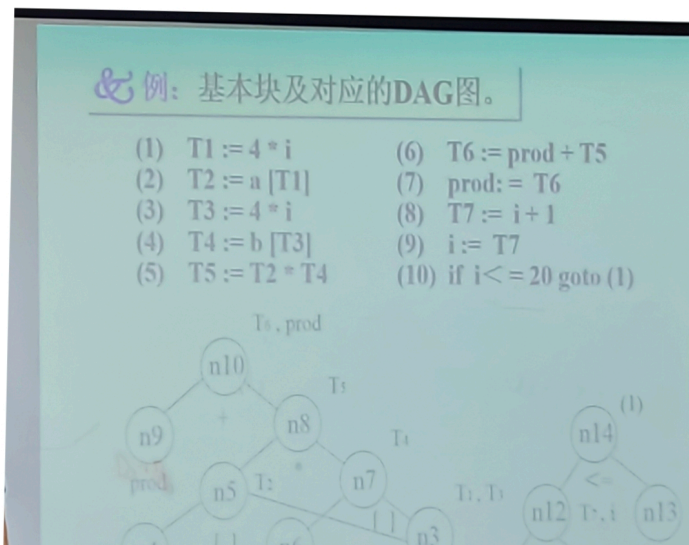


总结:

1. 文法给你, 让你验证句子 | 推导方法.
2. 文法给你, 产生式给你, 画语法树. 语法树所有的短语(素短语, 直接短语) 都是短语. 叶子结点, 子树都为短语. 最左边叶子结点为直接.
3. 判断二义性.
4. 抽象语法树, DAG图. 根据表达式画(优化, 中间代码画, DAG图, 例题)
5. NFA, 正规式, 非确定性有限
6. 文法给你, 画正则文法, 画状态图
7. 判断是否符合状态图.
8. 消除左递归
9. 优化, 中间代码画, DAG图, 例题) ✓
在DAG图, 中间代码有优化, 一章

大题总结:

1. First Follow
2. 递归优化.
3. 推导(左右)
4. 给表达式画三元/四元式



1. 总结:

第二章 文法与形式语言简介

- (1) 给出句型或句子最左推导或最右推导(规范推导);
- (2) 画出句型或句子的语法树;
- (3) 求句型的短语、简单短语、句柄;
- (4) 判断一个文法是二义性的文法

第三章 词法分析

1、正规文法和有限自动机的等价性

NFA $\rightarrow$ DFA $\rightarrow$ 最小

2、正规式和有限自动机的等价性

3、NFA到DFA转换的子集法及最小化

作业:

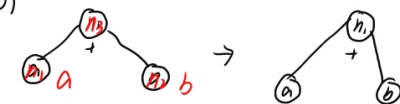
语法分析
词法分析

$(a+b)*c$ $7, * c$

三元 $(+, a, b)$ $(*, 7, c)$

四元 $(+, a, b, 7)$ $(*, 7, c, 7_2)$

DAG图: 有向非循环图 / 抽象语法树
 $(a+b)$



有11看* 不考.

目标代码形式

简答 $\Rightarrow$ 汇编程序最终目标代码有什么? 机器语言, 汇编代码

- 局部优化 编译程序
- 循环优化 编译与解释的区别
- 上下文法
- 有限自动集
- 解释文法
- 算法优先分析

第四章 自顶向下的语法分析

- (1) 消除直接左递归
- (2) 消除间接左递归的算法
- (3) First集合和Follow集合的求解方法
- (4) 避免回溯的判断方法
- (5) 简单回溯的消除方法
- (6) LL(1)分析表的构造算法
- (7) LL(1)分析方法

左递归 $\rightarrow$ First $\rightarrow$ select $\rightarrow$ Follow
LL(1)
 $\downarrow$
分析

第5章 LR分析法

1、LR(0)分析法 ✓

新题

2、SLR(1)分析法 ✓

3、LR(1)分析法 ✓

4、LALR(1)分析法 ✓

语法树

抽——→ 具体的

最左终结推导

最右推导

eg: 句子 $-(id + id)$

最左推导

$E \rightarrow -E$

$E \rightarrow -(E)$

$E \rightarrow -(id)$ 不能推

$\therefore E \rightarrow -(E + E)$

$E \rightarrow -(id + E)$

$E \rightarrow -(id + id)$

从一个句型到另一个句型的推导往往不唯一。
最左推导: 任何一步 $\alpha \Rightarrow \beta$ 都是对 α 中的最左非终结符进行替换。
最右推导: 任何一步 $\alpha \Rightarrow \beta$ 都是对 α 中的最右非终结符进行替换。

$E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid id$

句子 $-(id + id)$

最左推导

$E \Rightarrow_{lm} -E \Rightarrow_{lm} -(E) \Rightarrow_{lm} -(E + E)$

$\Rightarrow_{lm} -(id + E) \Rightarrow_{lm} -(id + id)$

最右推导 (规范推导)

$E \Rightarrow_{rm} -E \Rightarrow_{rm} -(E) \Rightarrow_{rm} -(E + E)$

$\Rightarrow_{rm} -(E + id) \Rightarrow_{rm} -(id + id)$

最右推导

$E \rightarrow (E)$

$E \rightarrow (id) \times$

$\therefore E \rightarrow E + E$

后接 $()$ 时为 $E + (E) \times$

所以

$E \rightarrow -E$

$E \rightarrow -(E)$

$E \rightarrow -(E + E)$

$E \rightarrow -(E + id)$

$E \rightarrow -(id + id)$

简单短语:

句柄:

上楼梯的方法 (从下到上从左到右)

例: 给定文法 $G[E]$:

$E \rightarrow T \mid E + T \mid E - T$

$T \rightarrow E \mid T * F \mid T / F$

$F \rightarrow i \mid (E)$

求句型 $(F+i) - T * (E-T)$ 的短语、简单短语和句柄。

解: 句型 $(F+i) - T * (E-T)$ 的分析树如图所示。

由此分析树可知, 该句型的短语有: $F, i, F+i, (F+i), E-T, (E-T), T * (E-T), (F+i) - T * (E-T)$ 。

简单短语有: $F, i, E-T$ 。

句柄为: F 。

1) 分析树

2) 找短语与句柄

'事情底层人做'

'安排听上面人的'

自底向上, 自左向右的

简单短语: 只有两层关系的

只有两层关系的叶子结点

的叶子结点

句柄: 找最左边的

F

二义性: 1) 推导 (2种)

2) 分析树 (2种不同)

都是一个方向上的

定义6: 如果一个文法的某个句子存在两棵不同的分析树, 则称这个文法是二义的。或者, 若一个文法的某个句子存在两个不同的最左 (右) 推导, 则称这个文法是二义的。

原因: 在产生句子的过程中某些直接推导有多于一种选择

一个句子有多于一棵分析树, 仅与文法和句子有关, 与采用的推导方法无关;

文法中缺少对文法符号优先级和结合性的规定

文法二义并不代表语言一定是二义的, 只有当产生一个语言的所有文法都是二义时, 这个语言才是二义的。

语法分析树

分析过程画成树

分析树

规则:

$G(E): E \rightarrow i \mid E + E \mid E * E \mid (E)$

$i * i + i$

$(i * i + i)$

最左:

$E \rightarrow (E)$

$E \rightarrow (E + E)$

$E \rightarrow (E * E + E)$

$E \rightarrow (i * E + E)$

$E \rightarrow (i * i + E)$

$E \rightarrow (i * i + i)$

最右:

$E \rightarrow (E)$

$E \rightarrow (E + E)$

$E \rightarrow (E + i)$

$E \rightarrow (E * E + i)$

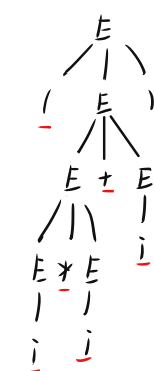
$E \rightarrow (E * i + i)$

$E \rightarrow (i * i + i)$



叶子结点
 $(i * i + i)$

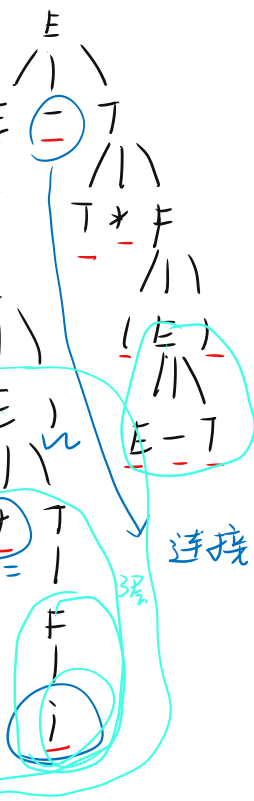
树的分析树不同但结果相同。



叶子结点
 $(i * i + i)$

$(F+i) - T * (E-T)$

最左:



叶子结点

所以 $(F+i) - T * (E-T)$

$E-T$

连接左右肩

最底层

$F \quad i$

至: $F, i, F+i, (F+i)$

第二层

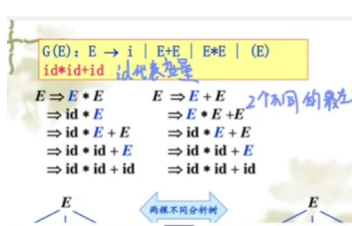
$F+i$

$(F+i)$

右: $E-T, (E-T), T * (E-T)$

所以

$(F+i) - T * (E-T)$



1) 先找到文法

$id * id + id$

最左:

$E \rightarrow E * E$

$\rightarrow id * E$

$\rightarrow id * E + E$

$\rightarrow id * id + E$

$\rightarrow id * id + id$

$E \rightarrow E + E$

$\rightarrow E * E + E$

$\rightarrow id * E + E$

$\rightarrow id * id + E$

$\rightarrow id * id + id$

中间的非终结符

证明二义性:

1. 已知文法 $G[S]: S \rightarrow aSb | Sb | b$, 证明此文法是二义性的。(8分)

$S \rightarrow aSb | Sb | b$

二义性: 1) 找句子 (不同结构)

2) 画树 } $\rightarrow$ 最左最右推导

2种 $\rightarrow$ 二义性

因为句子 'abbb' 的最左推导有 2 个语法分析树

对于句子 abbb 有 2 种不同的最左推导,

该文法为二义性文法

1. (10分) 对于文法 $G:$

$S \rightarrow aSbS | aS | d$

证明该文法是二义性文法。

$S \rightarrow aSbS | aS | d$

aadbd 句子的最左推导有 2 个语法分析树

所以该文法是二义性文法

2. 已知文法 $G[S]: S \rightarrow aAcB | Bd \quad A \rightarrow AaB | c \quad B \rightarrow bScA | b$

① 画出句子 acabcbbdcc 的语法树。(4分)

② 列出句子 acabcbbdcc 的所有短语、简单短语和句柄。(4分)

①

② 所有短语: c, b, cab, acabc, b, bd, c, bbdcc.

从下往上, 从左向右

所以 acabcbbdcc

直接短语: cbbc (2层的) 叶子结点

句柄: 最左: a

上级只能有 2 层

五、综合题 (共 40 分)

1. (10分) 对于文法 $G[S]:$

$S \rightarrow 1A | 0B | \epsilon \quad A \rightarrow 0S | 1AA \quad B \rightarrow 1S | 0BB$

(1) (3分) 请写出三个关于 $G[S]$ 的句子;

(2) (4分) 符号串 11A0S 是否为 $G[S]$ 的句型? 试证明你的结论。

(3) (3分) 试画出 001B 关于 $G[S]$ 的语法树。

1) 1100

0011

10

01

2) 11A0S

3) 001B

NFA, DFA, 最小化 DFA

1) 词法分析

正规式 $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ 最小化

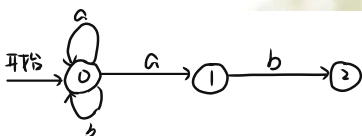
构造与正规式 $b^*abb^*(abb)^*$ 等价的状态最少的 DFA。

2) 画出 NFA

手工构造 $\rightarrow$ 不确定的有限自动机

不确定性

<2> 单词在 NFA 中的表现 (NFA 如何识别单词)
对字符串, 从初态开始, 经过一系列状态转移, 最后到达终态。
例如: 对于字符串 ab 有
定义: $\delta(0,a)=1, \delta(1,b)=2$
转换矩阵: $m[0,a]=\{0,1\}, m[1,b]=2$
转换图: $0a1b2$
显然, 转换图最直观, 即每一个单词, 实质上是从初态开始到某个终态的路径上的标记。



NFA 初态 $\rightarrow$ 终态
反复试探

<3> NFA (识别记号) 的特点

NFA 识别记号的最大特点是它的 **不确定性**, 即在当前状态下对同一字符有 **多于一个** 的下一状态转移。

<4> 不确定性的表现

定义: δ 函数是 1 对多的;
转换图: 同一状态有多于一条边标记相同字符转移到不同的状态;
转换矩阵: $M[s,a]$ 是一个状态的集合。

<7> NFA 识别单词存在的问题

- (a) 只有尝试了全部可能的路径, 才能确定一个输入序列不被接受, 而这些路径的条数随着路径长度的增长成指数增长。
- (b) 识别过程中需要进行大量回溯, 时间复杂度升高且算法趋于复杂。

确定化的过程

1) 画图:

正规式 $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ 最小

闭包
连接
或

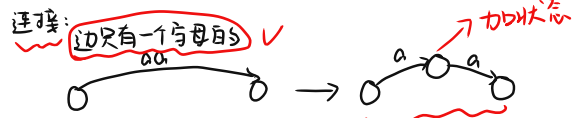
例2: 识别所有含相继两个 a 或相继两个 b 的字

① 画图 $(a|b)^*(aa|bb)(a|b)^*$ 或 $(a|b)^*aa(a|b)^* \cup (a|b)^*bb(a|b)^*$

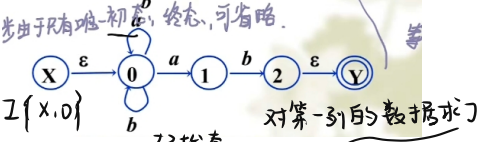
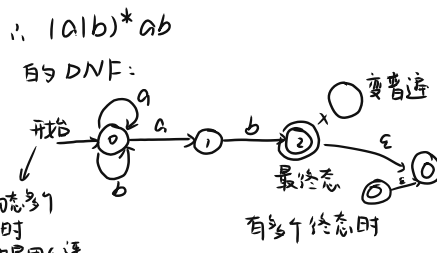
$(a|b)^*$ 闭包 a^* : a 可能有无数个
 b^* : b 可能有无数个

而 $a|b$ 是或的关系所以

ϵ^* 闭包 ⑤ - 一直是自己
什么都不做



组合: 可用 ϵ 新状态 (可省略)



2) 确定化

Δ	ΔI_a	I_b
初态, ϵ 闭包	根据输入个数确定到数	
$\{x, 0\}$ x 是 ϵ 的闭包 唯一起始	$\{0, 1\}$ x 输入 a x 输入 b $\{x, 0\} \xrightarrow{a} \{0, 1\}$ $\{x, 0\} \xrightarrow{b} \{0, 1\}$	$\{0\}$ D 输入 a 可能 D 输入 b 可能

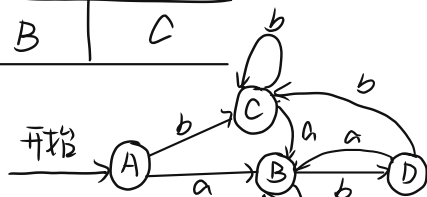
I_a 与 I_b 与开始的 I 不同 $I_a, I_b = I$
重复开始 $\begin{cases} I_a \{0, 1\} & \{0, 1\} & \{0, 2, Y\} \text{ 等于 } \rightarrow \text{结束} \\ I_b \{0\} & \{0, 1\} & \{0\} \text{ 不等于 } \rightarrow \text{移互边重复} \end{cases}$

确定化的过程

I	I_a	I_b
$\{X, 0\}$ 闭包	$\{0, 1\}$	$\{0\}$
$\{0, 1\}$	$\{0, 1\}$	$\{0, 2, Y\}$
$\{0\}$	$\{0, 1\}$	$\{0\}$
$\{0, 2, Y\}$	$\{0, 1\}$	$\{0\}$

DFA 的表

状态	输入符号	
	a	b
A	B	C
B	B	D
C	B	C
D	B	C



从正规式建立识别器的步骤:

- 从正规式构造 NFA
- 把 NFA 变成 DFA
- 将 DFA 化简
- 从 DFA 构造词法分析器



不确定的有限自动机

例 在如图所示的 NFA 上识别输入序列 abb 和 $abab$

abb: 0 $\xrightarrow{a}$ 1 $\xrightarrow{b}$ 2 $\xrightarrow{b}$ 3 (终态, 接受)

abab: 0 $\xrightarrow{a}$ 1 $\xrightarrow{b}$ 2 $\xrightarrow{a}$ 0 $\xrightarrow{b}$ 3 (终态, 接受)

abb: 0 $\xrightarrow{a}$ 1 $\xrightarrow{b}$ 2 $\xrightarrow{b}$ 3 (终态, 接受)

abab: 0 $\xrightarrow{a}$ 1 $\xrightarrow{b}$ 2 $\xrightarrow{a}$ 0 $\xrightarrow{b}$ 3 (终态, 接受)

练习 3.6.3: 使用算法 3.25 和 3.20 将下列正则表达式转换成 DFA。

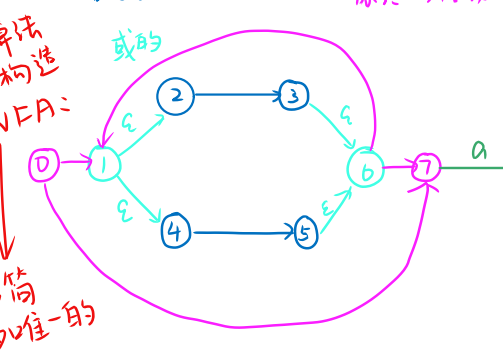
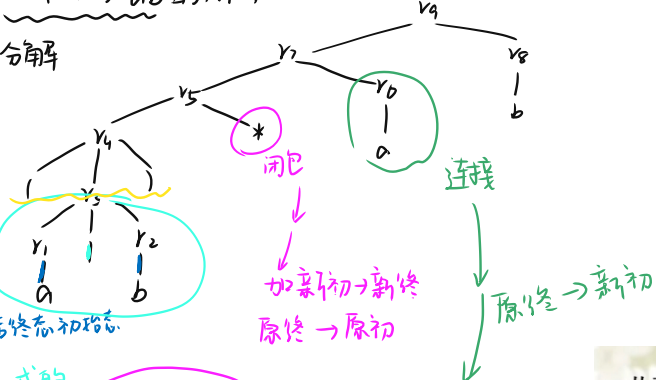
- $(a|b)^*$
- $(a^*|b^*)^*$
- $((\epsilon|a)b^*)^*$
- $(a|b)^*abb(a|b)^*$

1) 方法

构造词法分析器的一般方法和步骤:

- <1> 用正规式对构词模式进行描述;
- <2> 为每个正规式构造一个 NFA, 它识别正规式所表示的正规集;
- <3> 将构造出的 NFA 转换成等价的 DFA, 这一过程也被称为确定化;
- <4> 优化 DFA, 使其状态数最少, 这一过程也被称为最小化;
- <5> 从优化后的 DFA 构造词法分析器。

$R = (a|b)^*ab$ 的 NFA



从正规式建立识别器的步骤:

- 从正规式构造 NFA
- 把 NFA 变成 DFA
- 将 DFA 化简
- 从 DFA 构造词法分析器

3例

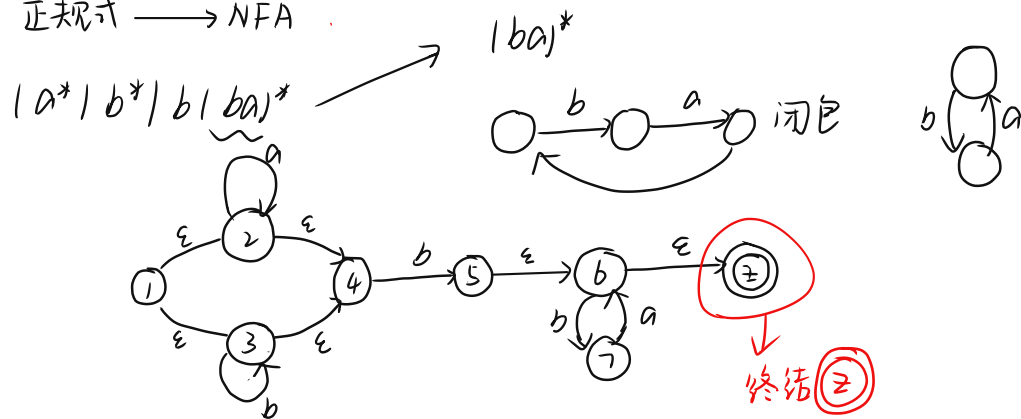
例题

一道第三章书

例：请构造与正规式 $(a^* | b^*)b(ba)^*$ 等价的状态最少的DFA。

(1)构造NFA如图所示：

1) 正规式 $\rightarrow$ NFA



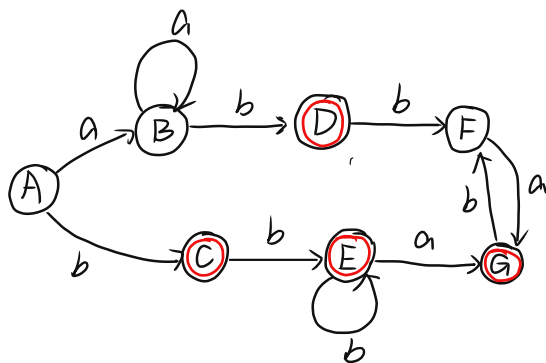
2) NFA $\rightarrow$ DFA:

I	Ia	Ib
A {1, 2, 3, 4}	B {2, 4}	C {3, 4, 5, b, z}
B {2, 4}	B {2, 4}	D {5, b, z}
C {3, 4, 5, b, z}		E {3, 4, 5, b, 7, z}
D {5, b, z}		F {7}
E {3, 4, 5, b, 7, z}	G {b, z}	E {3, 4, 5, b, 7, z}
F {7}	G {b, z}	
G {b, z}		F {7}

含z的都是终态和z一样

CDEG 终态

3) 画状态图 DFA



4) 画简DFA

终点与非终点画成2个集合

$$F_1 = \{A, B, F\} \quad F_2 = \{C, D, E, G\}$$

$$F_{11} = \{A, B\} \quad F_{12} = \{F\} \quad F_2 = \{C, D, E, G\}$$

$$F_2 = \{C, D, E, G\}$$

$$F_{21} = \{C, D, G\} \quad F_{22} = \{E\}$$

$$F_{11} = \{A, B\} \quad F_{12} = \{F\}$$

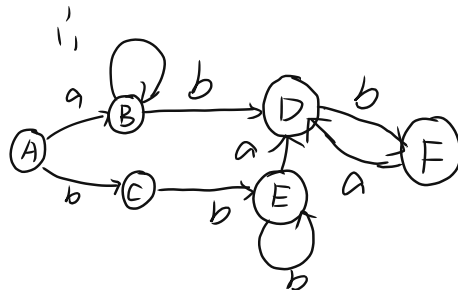
$$F_{21} = \{C, D, G\}$$

$$\checkmark F_{212} = \{D, G\} \quad \checkmark F_{11} = \{A, B\}$$

$$F_{22} = \{E\} \quad F_{12} = \{F\}$$

$$F_{11} = \{A, B\}$$

$$F_{111} = \{A\} \quad F_{112} = \{B\}$$



17

Handwritten notes and diagrams related to DFA construction and minimization. Includes a small DFA diagram and a table of sets.

$\{E, F, G, z\}$	$\{H\}$	$\{E, F, G, z\}$
$\{I, J, G, z\}$	$\{H\}$	$\{J, G, z\}$
$\{J, G, z\}$	$\{H\}$	$\{J, G, z\}$

① 行坐标: I, I输入符号(输入)
② (第一个I) I只在第一个位置, (开始符号)求空包,
③ I先输入符号, I里空包用完, 求空包扩充自己.
④ I只在第一个位置, (开始符号)求空包, 其它从右边来(从右到左)
⑤ 在右边的集合, 若已在I中出现, 便不再加入I
⑥ 走到右边再也没有新的I出现 $\rightarrow$ 结束

2)

1. 给I标号(从1开始)
2. 给右边I输入找对象
3. 画圈, ①, 开始
4. 终止符号.

3) 最小化

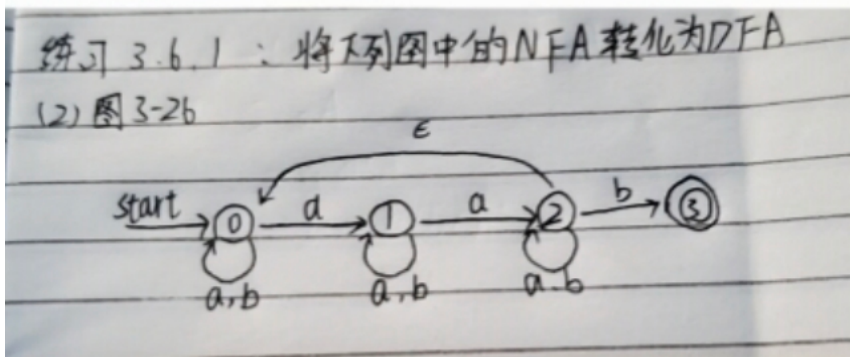
找判链 留相同 找不出后最后最小化结束

$K1 = \{1, 2, 3, 5\}$ $K2 = \{4, 6, 7, 8\}$
状态2和5无a输入, 状态1和状态3有a输入, 因此将K1分割成
 $K11 = \{1, 3\}$ $K12 = \{2, 5\}$

共5页, 第4页

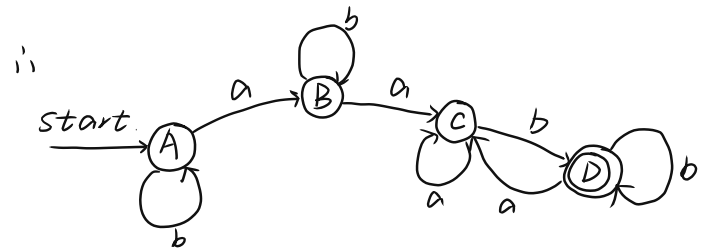
《编译原理》模拟题参考答案 软件学院

进一步可看出状态1和状态3是等价的, 而状态2和状态5可能等价:
由于状态4、6、7、8输入a都到达K12, 输入b都到达K2, 所以状态4、6、7、8等价, 也进一步说明状态2、5等价。
这样, 原DFA的状态集合被划分为: $\{1, 3\}, \{2, 5\}, \{4, 6, 7, 8\}$

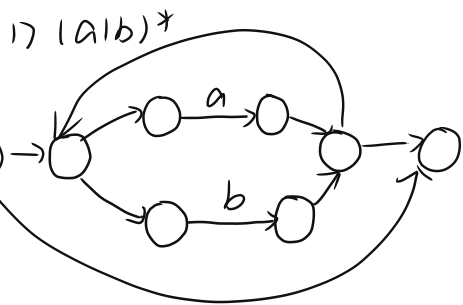
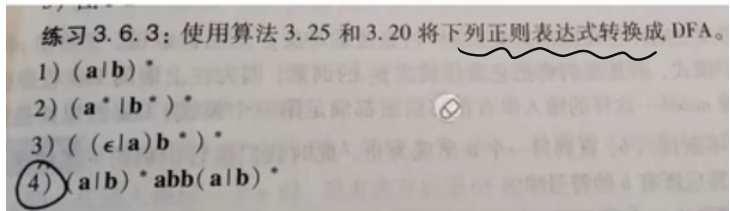


NFA 转换为 DFA

I I_a I_b
 $A \{0\}$ $B \{0, 1\}$ $A \{0\}$
 $B \{0, 1\}$ $C \{0, 1, 2\}$ $B \{0, 1\}$
 $C \{0, 1, 2\}$ $C \{0, 1, 2\}$ $D \{0, 1, 2, 3\}$
 $D \{0, 1, 2, 3\}$ $C \{0, 1, 2\}$ $D \{0, 1, 2, 3\}$ D 为终态



已经最简



消除左递归:

一般而言, 假定关于 P 的全部产生式是

$$P \rightarrow P\alpha_1 | P\alpha_2 | \dots | P\alpha_m | \beta_1 | \beta_2 | \dots | \beta_n$$

其中, 每个 α 都不等于 ϵ , 每个 β 都不以 P 开头

那么, 消除 P 的直接左递归就是把这些规则改写成:

$$P \rightarrow \beta_1 P' | \beta_2 P' | \dots | \beta_n P' | \alpha_1 P' | \alpha_2 P' | \dots | \alpha_m P' | \epsilon$$

左递归变右递归

$P' \rightarrow \alpha P' | \epsilon$

$P \rightarrow \alpha, P'$

二、综合题 (每小题 15 分, 共 30 分)

1. 已知文法 $G: S \rightarrow aSb | P \quad P \rightarrow bPc | bQc \quad Q \rightarrow Qa | a$

① 给出消除左递归, 提取左公因子之后的文法。(2 分)

② 求出文法中每个非终结符的 First 集和 Follow 集。(6 分)

③ 构建 LL(1) 分析表。(7 分)

	First	Follow
S	$\{a, b\}$	$\{\#, b\}$
P	$\{b\}$	$\{\#, b, c\}$
P'	$\{b, a\}$	$\{\#, b, c\}$
Q	$\{a\}$	$\{c\}$
Q'	$\{a, \epsilon\}$	$\{c\}$

LL(1) 分析表

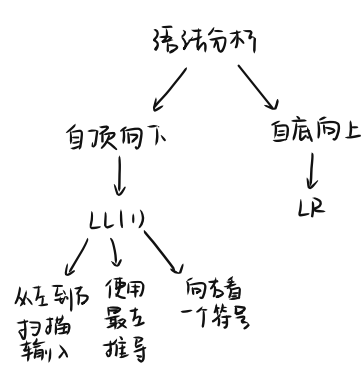
17. 行是所有的终结符
13 位 #)

填表

	a	b	c	#	非终结符
S	$S \rightarrow aSb$	$S \rightarrow P$			
P		$P \rightarrow bP'$			
P'	$P' \rightarrow aQ'$	$P' \rightarrow bP'$	$P' \rightarrow c$		
Q	$Q \rightarrow aQ'$				
Q'	$Q' \rightarrow aQ'$				

含 ϵ 的式子, 在 Follow 位置, 加上

四章:



First集: 求开头 可以有空

例4.1 已知文法G[E]:

1) 非终结符列出来, 从左边看

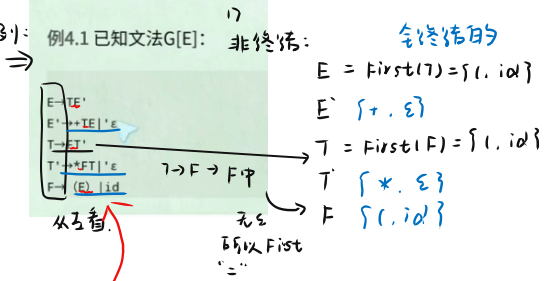
2) 先推算是终结符的

3) 再看是非终结符的

无ε: First = ""

有ε: 加下一个

注意: 先做左递归, 提左公因式, 再做First



例4.5 给定文法G[E]:

非终结	终结	非终结
E → TE' ε	E	{ (, i }
E' → +TE' ε	E'	{ ε, +, - }
T → ET' ε	T	{ (, i }
T' → *ET' ε	T'	{ ε, *, / }
F → (E) ε	F	{ (, i }
A → + -	A	{ +, - }
M → * /	M	{ *, / }

例: G[S]: S → aH

H → a | d

M → Ab

M → ε

A → aM

A → ε

First集: First(S) = { a }

First(H) = { a, d }

First(M) = { a, ε }

First(A) = { a, ε }

Follow集: Follow(S) = { # }

Follow(H) = { # }

Follow(M) = { d, b }

Follow(A) = { b }

例: G[E]: E → TE' | ε

E' → +E' | ε

T → ET' | ε

T' → *T' | ε

F → (E) | ε

P → (E) | a | b | ε

First集: First(E) = { (, a, b, ε }

First(E') = { +, ε }

First(T) = { (, a, b, ε }

First(T') = { *, ε }

First(F) = { (, a, b, ε }

First(P) = { (, a, b, ε }

Follow集: Follow(E) = { #,) }

Follow(E') = { #,) }

Follow(T) = { +, #,) }

Follow(T') = { +, #,) }

Follow(F) = { (, a, b, ε, +, #,) }

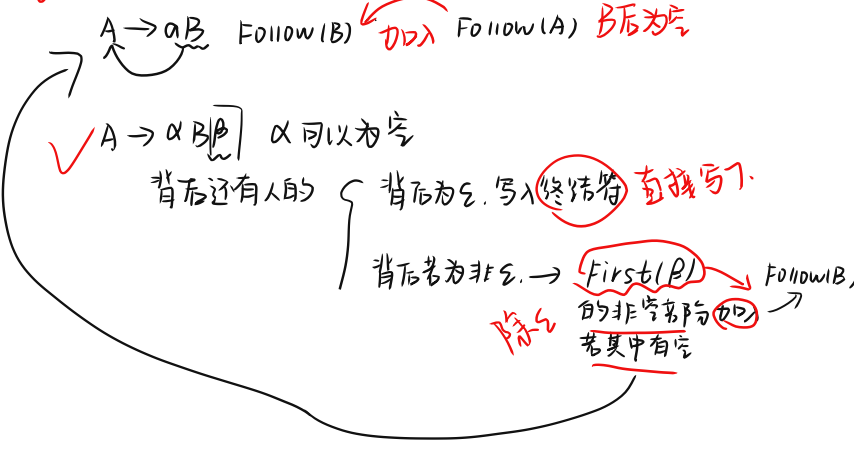
Follow(P) = { *, (, a, b, ε, +, #,) }

如何求FIRST集和FOLLOW集 (first集看产生式左边 follow集看产生式右边)

- ① 文法开始符, 必有 #
- ② A → αB FOLLOW(B) B后为ε, 将 follow(A) 加入到 follow(B) 中
- ③ A → αBβ { β是终结符, 直接写T集; β是非终结符, first(β) 加入到 follow(B) 中. 如果 β → ε, 带入 A → αBβ 中, 得到 A → αB

Follow集: 求后-1(没有空)

- 1) 把非终结符列出来, 从右边看
- 2) 文法开始符, 必有 #
- 3) 后面没有路的, 回头找妈



消除左递归

形式: $A \rightarrow A\alpha | \beta$ (α, β 为终结符串)

改为: $\begin{cases} A \rightarrow \beta A' \\ A' \rightarrow \alpha A' | \epsilon \end{cases}$

eg: G[E]: $E \rightarrow E+T | T$

$T \rightarrow T*F | F$

$F \rightarrow (E) | i$

eg: $E \rightarrow E+T | E-T | T \Rightarrow \begin{cases} E \rightarrow TE' \\ E' \rightarrow +TE' | -TE' | \epsilon \end{cases}$

消除回溯

形式: $(A \rightarrow \underline{\beta_1} | \underline{\beta_2} | \dots | \underline{\beta_i} | \beta_{i+1} | \dots | \beta_j)$

改为: $\begin{cases} A \rightarrow \underline{\beta_1} A' | \beta_{i+1} | \beta_j \\ A' \rightarrow \beta_1 | \beta_2 | \dots | \beta_i \end{cases}$

eg: G[A]: $A \rightarrow aAB | a$

$B \rightarrow bB | d$

回溯: $A \rightarrow aA'$

$A' \rightarrow AB | \epsilon$

左递归: $B \rightarrow dB'$

$B' \rightarrow bB'$

LL(1)文法

- 1. 消除左递归
- 2. 消除回溯
- 3. 改造后的文法
- 4. First:
- Follow:
- 5. LL(1)分析表:

例:
<3.10> 已知文法 $G(A)$:
 $A \rightarrow aAB \mid a$
 $B \rightarrow Bb \mid d$
<1> 试给出与 $G(A)$ 等价的 LL(1)文法 $G'(A)$;

<3.11> 将下述文法改造为 LL(1)文法: $G[V]: V \rightarrow N \mid N[E]$
 $E \rightarrow V \mid V+E$
 $N \rightarrow i$

改造后的文法 $G'(A)$:
 $A \rightarrow aA'$
 $A' \rightarrow AB \mid \epsilon$
 $B \rightarrow dB'$
 $B' \rightarrow bB' \mid \epsilon$
可以推出两种情况
的
 $A \rightarrow a \mid \epsilon$
 $First(A) \cap First(B) = \emptyset$
说明为 LL(1)
若 $B = \epsilon$
 $First(A) \cap Follow(A) = \emptyset$
 $Follow(A) = \{ \epsilon \}$
 $First(A) \cap Follow(A') = \emptyset$
 $\therefore$ 为 LL(1)

判断
 $V \rightarrow N \mid N[E]$ $First(\epsilon) \cap First(C) = \emptyset$ $First(C) \cap Follow(V)$
 $E \rightarrow V \mid V+E$ $First(\epsilon) \cap First(+)=\emptyset$ $First(+)\cap Follow(E)$
 $\therefore$ 为 LL(1) 文法

构造 $G(A')$ 的 LL(1) 分析表:

	a	b	d		#
A	$A \rightarrow aA'$				
A'	$A' \rightarrow AB$		$A' \rightarrow \epsilon$		$A' \rightarrow \epsilon$
B			$B \rightarrow dB'$		
B'		$B' \rightarrow bB'$		$B' \rightarrow \epsilon$	

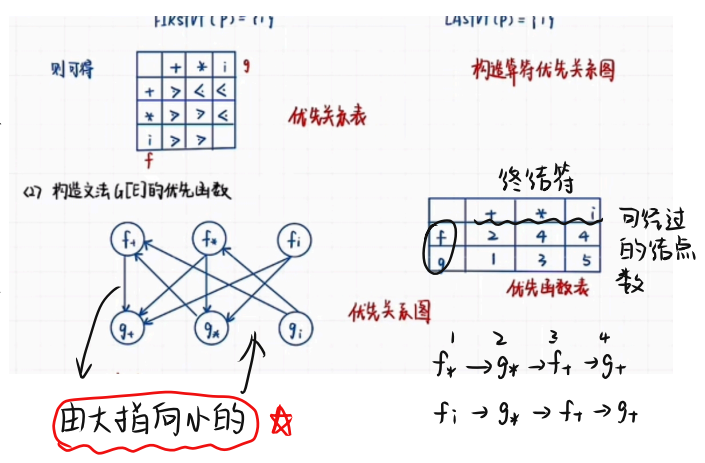
给出输入串 $aad| \#$ 的分析过程

<3> 给出输入串 $aad| \#$ 的分析过程

符号栈	当前输入符号	输入串
#A	a	aad #
#A'a	a	aad #
#A'	a	aad #
#BA	a	aad #
#BA'a	a	aad #
#BA'	d	aad #
#B	d	aad #
#B'd	d	aad #

优先关系图:

优先函数表



算符优先文法:

$E \rightarrow E+T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid i$

firstvt: $E = \{ +, *, (, i \}$
 $T = \{ *, (, i \}$
 $F = \{ (, i \}$
lastvt: $E = \{ +, *,), i \}$
 $T = \{ *,), i \}$
最后一个终结符: $F = \{), i \}$

优先关系表

	+	*	(	)	#
+	>	<	<	>	>
*	>	>	<	>	>
(	>	>		>	>
)	<	<	<	=	=
#	>	>		>	

2个#之间的
关系
 $\# = \#$
 $\# < Firstvt(E)$
 $lastvt > \#$

5章:

1) LR(0)分析法:

DFA图

拓广文法 → 列项目 → 项目集规范族 → 分析表

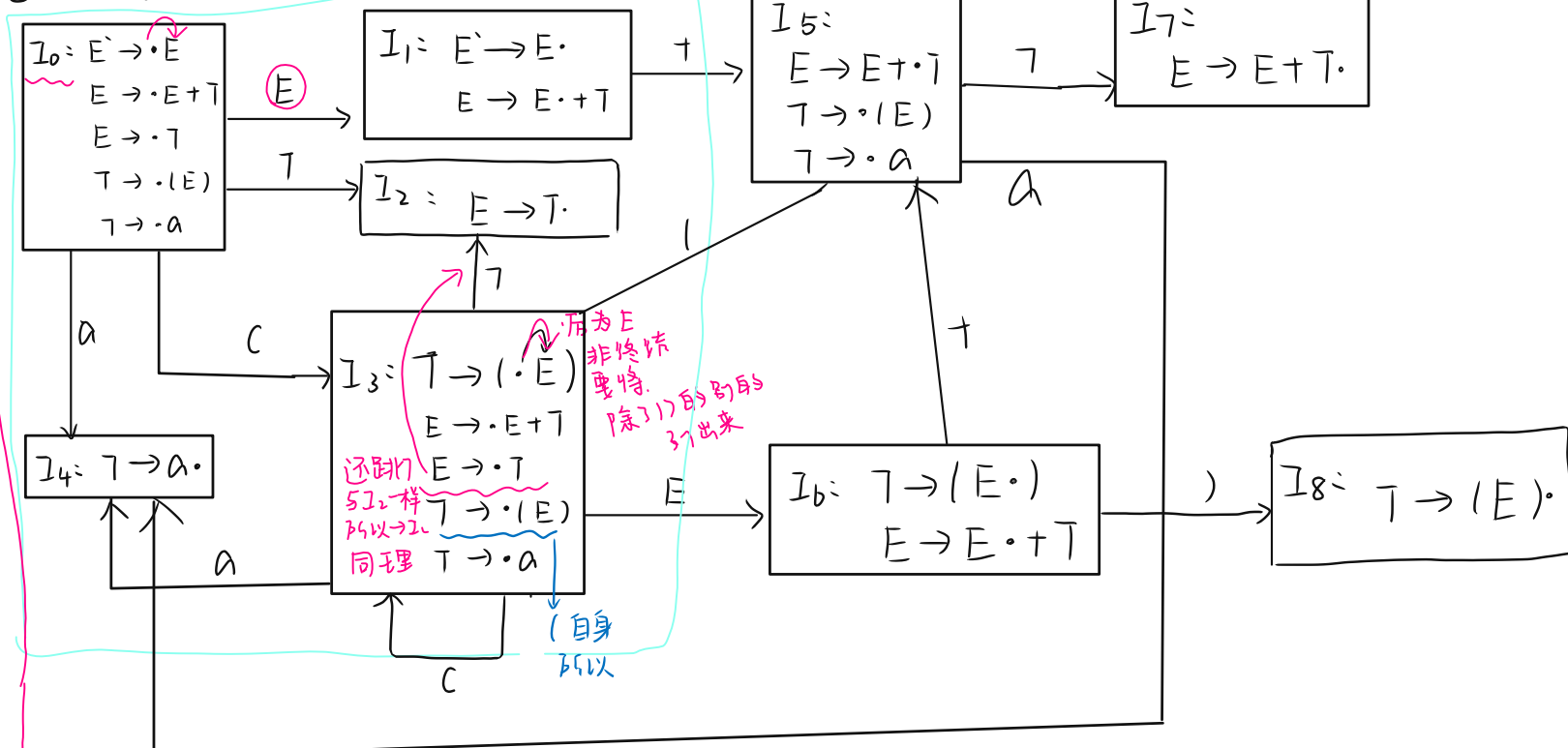
① → ② → ②

<3.19> 试构造下述文法的LR(0)分析表: $G[E]: E \rightarrow E+T \mid T$
 $T \rightarrow (E) \mid a$

- ① $G[E]$ (0) $E' \rightarrow E$
(1) $E \rightarrow E+T$
(2) $E \rightarrow \cdot T$
(3) $T \rightarrow (\cdot E$
(4) $T \rightarrow a \cdot$

20分析完分析1

② 项目规范族



ACTION GOTO 表

终结符+ #

ACTION 所有终

GOTO 非终结符

状态	a	+	(	)	#	E	T
0	S ₄		S ₃			1	2
1		S ₅			acc		
2	r ₂	r ₂	r ₂	r ₂	r ₂		
3	S ₄		S ₃			6	2
4	r ₄	r ₄	r ₄	r ₄	r ₄		
5	S ₄		S ₃				7
6		S ₅		S ₈			
7	r ₁	r ₁	r ₁	r ₁	r ₁		
8	r ₃	r ₃	r ₃	r ₃	r ₃		

有 $I_0 \rightarrow$ 就 $I_1 \sim I_n$

acc 状态:

在拓广文法

(0) $E' \rightarrow E$

所以在 I_1 中并写 acc

SLR(1)分析表
与LR(0)分析表+分像, 只是多求一步 FOLLOW 表

为了解决LR(0)中的问题 1, 2

1. 移进 —— 归约
2. 归约 —— 归约

16. 活前缀定义, 项目定义, 会识别四种项目(移进, 归约, 接受, 待约)

规范句型的一个前缀(不含句柄之后的任何符号)

文法G[S]中每一个产生式的右部添加一个圆点“·”

·后面是终结符

① 归约项目 ($A \rightarrow \alpha \cdot$) ·在最后 ③ 移进项目 ($A \rightarrow \alpha \cdot X \beta$)

② 接受项目 ($S \rightarrow \alpha \cdot$) 开始文法对应的 ④ 待约项目 ($A \rightarrow \alpha \cdot X \beta$) ·后面是非终结符

状态	ACTION					非终	
	a	+	(	)	#	E	T
0	S ₄		S ₃			1	2
1		S ₅			acc		
2	r ₂	r ₂	r ₂	r ₂	r ₂		
3	S ₄		S ₃			6	2
4	r ₄	r ₄	r ₄	r ₄	r ₄		
5	S ₄		S ₃				7
6		S ₅		S ₈			
7	r ₁	r ₁	r ₁	r ₁	r ₁		
8	r ₃	r ₃	r ₃	r ₃	r ₃		

把要写V的先不写

将文法的Follow集求出

$r_2 \rightarrow I_2: E \rightarrow T \cdot$

把 FOLLOW 中的 E

<3.20> 试构造下述文法的 SLR(1) 分析表: $G[E]: E \rightarrow E+T \mid T$, 该文法能否构造
拓广文法为 $G'[S']$:

(0) $S' \rightarrow E$

(1) $E \rightarrow E+T$

(2) $E \rightarrow T$

(3) $T \rightarrow F*$

(4) $T \rightarrow F$

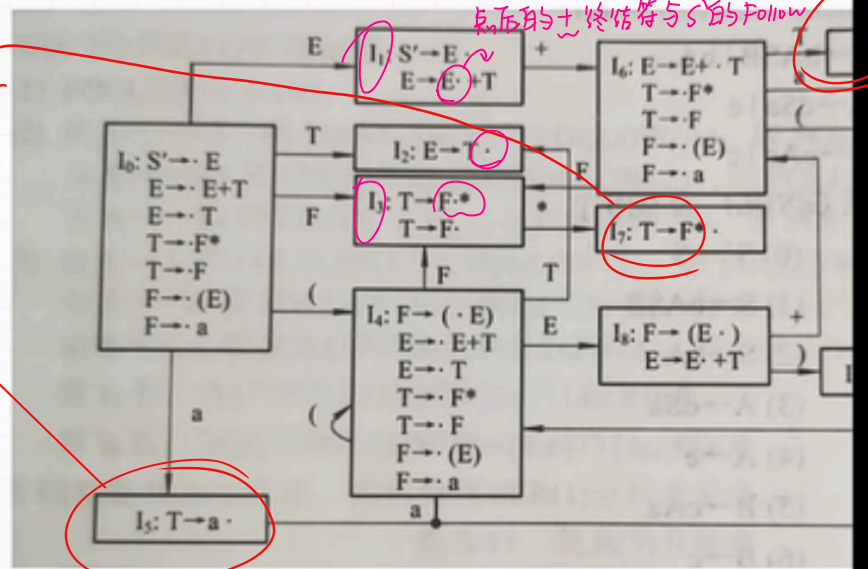
(5) $F \rightarrow (E)$

(6) $F \rightarrow a$

$T \rightarrow F* \mid F$
 $T \rightarrow (E) \mid a$

求门集 若为0

后面的+ 终结符与S的Follow



○ ○ ○

r₆ r₆ r₆ r₆

r₃ r₃ r₃ r₃

r

LR(1)分析表
 拓广文法 — 项目集规范族 — LR(1)分析表
 (带向前搜索符)

eg1: 试构造该文法的LR(1)分析表
 $G[S]: S \rightarrow BB$
 $B \rightarrow aB \mid b$ 如何写向前搜索符

1) 拓广文法
 $G[S]: S' \rightarrow S$ Follow(S) = {#}
 $S \rightarrow BB$ First(B) = {a, b}
 $B \rightarrow aB$
 $B \rightarrow b$

① 文法开始符 Follow集
 ② 先看前再看后
 $A \rightarrow \alpha \cdot B \cdot a$ 任意
 $B \rightarrow$ 表达式 β 为字, 照抄 $B \rightarrow$ 表达式, a
 β 不为字, first(β) 加入, 后面
 (β) 为非终结符号
 β 为终结符号

2) (带向前搜索符) 项目集规范族

$I_0: S' \rightarrow \cdot S \cdot \#$
 $S \rightarrow \cdot BB \cdot \#$
 $B \rightarrow \cdot aB \cdot a \mid b$
 $B \rightarrow \cdot b \cdot a \mid b$

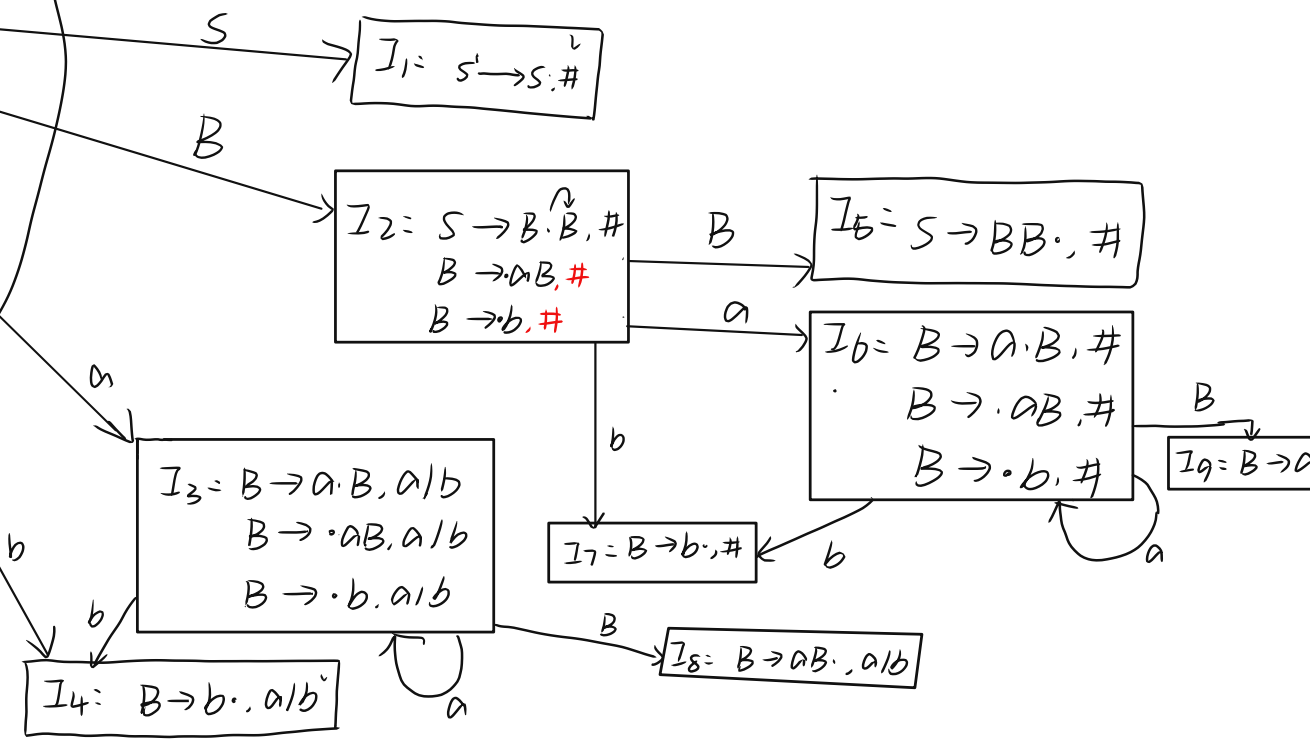
3.26 给出文法 $G[S]$ 及图 3-47 所示的 LR(1) 项目集规范族中的 0、1、2、3、4。
 $G[S]: S \rightarrow S; B \mid B$
 $B \rightarrow BaA \mid A$
 $A \rightarrow b(S)$

拓广文法:
 $G[S]: S' \rightarrow S$
 $S \rightarrow S; B$
 $S \rightarrow \cdot B$
 $B \rightarrow BaA$
 $B \rightarrow \cdot A$
 $A \rightarrow b(S)$
 $A \rightarrow \cdot b(S)$

2) 带向前搜索符 (项目集规范族)

$I_0: S' \rightarrow \cdot S \cdot \#$
 $S \rightarrow \cdot S; B \cdot \# \mid$
 $S \rightarrow \cdot B \cdot \# \mid$
 $B \rightarrow \cdot BaA \cdot \# \mid a$
 $B \rightarrow \cdot A \cdot \# \mid a$
 $A \rightarrow b(S) \cdot \# \mid a$

$G[S]: S \rightarrow S$ $I_0: S \rightarrow \cdot S \cdot \#$
 $S \rightarrow A$ $S \rightarrow \cdot A \cdot \#$
 $A \rightarrow BA$ $A \rightarrow \cdot BA \cdot \#$
 $A \rightarrow \epsilon$ $A \rightarrow \cdot \epsilon \cdot \#$
 $B \rightarrow aB$ $B \rightarrow \cdot aB \cdot a \mid b \mid \#$
 $B \rightarrow b$ $B \rightarrow \cdot b \cdot a \mid b \mid \#$
 $first(A) = \{a, b, \epsilon\}$ 带 λ
 $A \rightarrow \cdot B \cdot \#$



	ACTION			GOTO	
	a	b	#	S	B
0	S ₃	S ₄		1	2
1			acc		
2	S ₆	S ₇			5
3	S ₃	S ₄			8
4	r ₃	r ₃			
5			r ₁		
6	S ₆	S ₇			9
7			r ₃		
8	r ₂	r ₂			
9			r ₂		

$S' \rightarrow S \cdot acc$

构造 LR(1) 分析表

扩充文法 $\longrightarrow$ 项目集规范族 $\longrightarrow$ LR(1) 分析表

(带向前搜索符)

合并同心集

表达式相同

向前搜索符不同

构造 LR(1) 分析表

扩充文法

项目集规范族

LR(1) 分析表

(带向前搜索符)

LR(0)

r_n

SLR(1)

eg1: 试构造该文法的 LR(1) 分析表

$G[S]: S \rightarrow BB$

$B \rightarrow aB \mid b$

解: (1) 扩充文法

(0) $S' \rightarrow S$

(1) $S \rightarrow BB$

(2) $B \rightarrow aB$

(3) $B \rightarrow b$

$\text{Follow}(S') = \{\#\}$

$\text{First}(B) = \{a, b\}$

如何写向前搜索符?

① 文法开始符 Follow 集

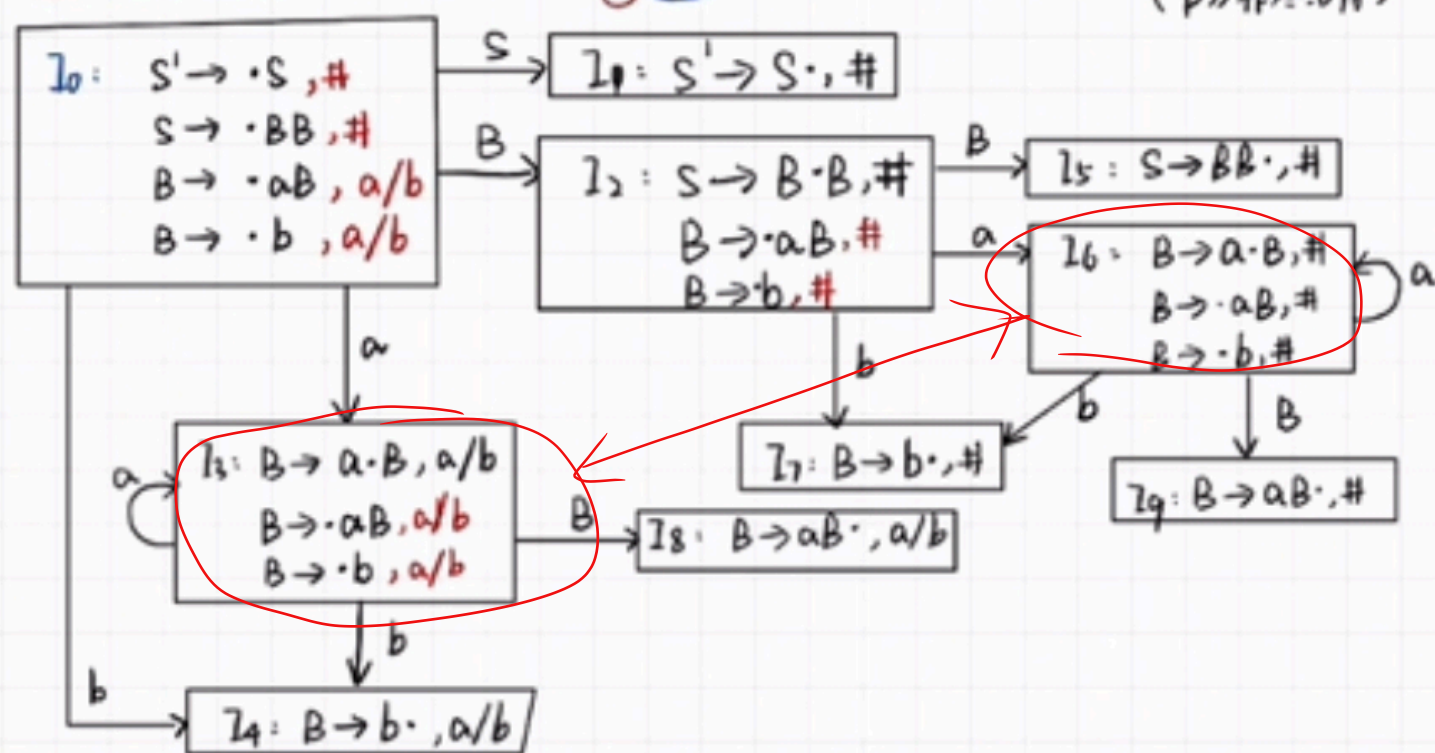
② "先看前面看后"

$A \rightarrow \alpha \cdot B \beta$ a $\begin{cases} \beta \text{ 为空, 照抄} \\ \beta \text{ 不为空, first}(\beta) \text{ 加入, 后面} \end{cases}$

$B \rightarrow \dots$

(β 为非终结符)

(2) (带向前搜索符) 项目集规范族



$I_3 I_6 = I_{36} = B \rightarrow a \cdot B, a/b \mid \#$

$B \rightarrow \cdot aB, a/b \mid \#$

$B \rightarrow \cdot b, a/b \mid \#$

$I_4 I_7 = I_{47} = B \rightarrow b \cdot, a/b \mid \#$

$I_8 I_9 = I_{89} = B \rightarrow aB \cdot, a/b \mid \#$

状态	ACTION			GOTO	
	a	b	#	S	B
0	S ₃₆	S ₄₇		1	2
1			acc		
2	S ₃₆	S ₄₇			5
36	S ₃₆	S ₄₇			89
47	r ₃	r ₃	r ₂		
5			r ₁		
89	r ₂	r ₂	r ₂		

